ECE-118/218 Introduction to Mechatronics

Lab 3 – Motors (and how to drive them)

University of California Santa Cruz — Spring 2026

Overview

In this lab you will learn to drive DC motors, RC servos, and stepper motors, and you will write the software needed to control them from the Uno32. Along the way you will explore PWM generation, inductive kickback and snubbing, unidirectional and bidirectional DC motor drive, and both direct and dedicated-board stepper control.

Lab at a Glance:

Part 0	Preparation for lab	Plan pin usage, power wiring, block diagrams, and a timer-driven PWM state-machine strategy before touching hardware.
Part 1	Driving an RC servo	Read a potentiometer, command a servo position, and mirror the input on the LED bar.
Part 2	Unidirectional DC motor drive	Use the DS3658 and a software PWM signal to control motor speed in one direction.
Part 3	Snubbing inductive kickback	Compare diode and Zener snubbing arrangements and study how the motor waveform changes.
Part 4	Bidirectional DC motor drive	Use an L298 H-bridge module to control DC motor speed and direction.
Part 5	Stepper motor control with an H-bridge	Drive a stepper directly through the L298 and compare full-step, wave, and half-step modes.
Part 6	Stepper motor with a dedicated driver	Control a stepper through the DRV8811 and explore performance limits with hardware assistance.

Comments: This lab introduces the motor-drive and high-current boards that you will later use in your project. They are reasonably robust, but no board is immune to abuse. High current mistakes are often fast, loud, and expensive.

You will be using the Uno32 stack, with the I/O board on top, the Uno32 in the middle, and a power-distribution board on the bottom. Power your circuits from the power-distribution board whenever possible because it is fused and is meant to protect you from common wiring mistakes.[a]

You will also be using ATX power supplies that provide 12V, 5V, and 3.3V rails. These supplies are clean and rugged, but they can source substantial current. The supply may protect itself; your wiring and boards may not.

Come into the lab with a plan for each part, a plan for the day, and a plan for the week. This lab rewards preparation more than any previous lab. If you try to do the preparation while simultaneously doing the build, you will take much longer and break more things.

Test hardware and software separately whenever you can. If you have spent more than about an hour on one task without real progress, stop and get another set of eyes on it.

It is worth making a cable that cleanly connects your boards to the Uno32 stack. This is not strictly required, but it saves a great deal of time because loose jumper-wire bundles are easy to miswire.

[a] At some point you will almost certainly wire something incorrectly and blow a fuse. **Slow down!**, find out why it happened, and fix the underlying mistake before applying power again.

©2026, Electrical and Computer Engineering Department, UC Santa Cruz *Compiled on: 20260425*

Checkoff: Check off each part with a tutor or TA as you complete it. It is strongly recommended that you check off each part before moving on to the next one, but there is no penalty for temporarily working out of order if staffing is busy. Just make sure you can still demonstrate the earlier work when asked.

Prelab

1. Choose a partner and join a group together in the "Lab 3 #XXX" category on CANVAS. Preferably work with someone you have not worked with before. See `ECE118_LabSubmission` for submission and group-management details.
2. Read the entire lab carefully and complete the required prelab work for each part. Submit your own individual prelab electronically as a PDF on CANVAS.

Each part below includes any additional prelab requirements for that section. Complete those exercises yourself, though you are welcome to discuss ideas with your teammate. Read carefully and answer everything that is asked.

Submission: Submit your prelab and lab report electronically in PDF format. See the lab submission document on the course website for the required naming and submission format, and verify that your files uploaded correctly.

Part 0 – Preparation for Lab

Overview

This lab requires considerably more preparation than the earlier ones if you want it to go smoothly. Before you start wiring motors and driver boards, work out the signal flow, power flow, pin assignments, and initialization steps for each part.

Reference Material

- ECE-118 Uno32 stack documentation
- Header files in the `C:/ECE118/include` directory
- `Stepper.h` in the `C:/ECE118/include` directory

Prelab

For each part of this lab, create a clear block diagram for that portion of the lab, see Fig. 1 for an example. The block diagram must clearly communicate:

- the Uno32 pins you will use, with names of the form "Port X Pin 3" or "X3"
- whether each Uno32 pin is acting as an input, output, or bidirectional signal
- the relevant power source for the circuit, such as the 12V rail, 5V rail, 3.3V rail, or the power-distribution board
- the type of signal each connection carries, such as analog voltage, PWM, direction, enable, or raw power
- the literal initialization code you expect to need for those ports and modules

On the same page as each block diagram, briefly explain why you chose those pins and what software setup is required. Keep the initialization snippets concrete: actual function calls, pin names, and semicolons, not prose descriptions.

Prelab Part 0

What one thing, and one thing only, should you ever insert directly into the 14-pin connectors on the boards?

Prelab Part 1

Draw a detailed state diagram for a hypothetical software PWM driver. Include an example plot of the expected output signal for a chosen frequency and duty cycle. The transitions between states should be triggered by timer events.

Prelab Part 2

Solder up a power connector for the Motor Drive boards: this will be banana plugs (red for power, black for ground), approximately 2 ft. of black/red power wire, and the female white plastic connector. Make sure the wires are well soldered, and that you have heat shrink over the ends where the joints to the connector are. You will use this connector during the project and will want to make sure it is well made.

Bumps and Road Hazards

Electronic Hygiene: This lab involves relatively high current. High current is dangerous both for you and for your hardware. Ten amps on the wrong wire can lift traces, destroy chips, melt insulation, and create a miserable debugging session. Good electronic hygiene is **critical**.

Never modify wiring with power applied. Triple-check polarity before connecting anything expensive or fragile. Keep power wiring color-coded, strain-relieved, and insulated with heat shrink. Use banana plugs for supply connections rather than improvised clips whenever possible.

Always inspect your boards for solder bridges, trimmed-wire fragments, or other conductive debris. Test new cables with a multimeter in continuity mode before using them. Cross-test them too: verify that the intended wire connects where it should and *does not* connect where it should not.

Avoid frustration-driven decisions. If you are tired, rushed, or angry, you are much more likely to make an expensive mistake. Pause, communicate with your partner, and ask for help early.

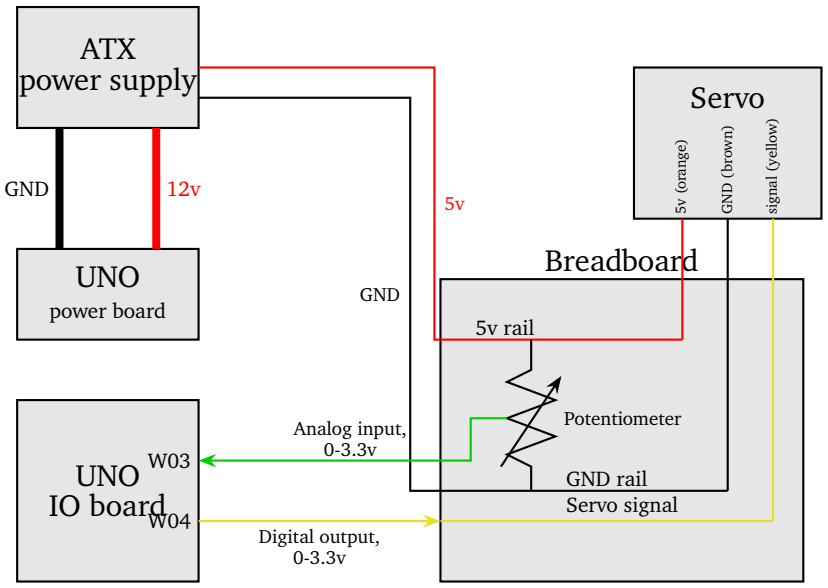


Figure 1: Example Block Diagram

Outline

Read through the entire lab and complete the drawings, block diagrams, and state diagrams requested above before building. Keep those documents for yourself even after submitting the prelab because you will refer to them repeatedly as you move between parts.

Lab Report: There is nothing additional to include in the final lab report for Part 0. This section is preparation work for the rest of the lab.

Part 1 – Driving an RC Servo

Overview

In this part you will drive a standard RC servo under software control. The servo uses a repeated pulse-width command signal, nominally at 20Hz, to represent position. You will read a potentiometer through the Uno32, scale that reading into a servo command, and display a scaled version of the potentiometer value on the LED bar.

Reference Material

- ECE118 Uno32 I/O board documentation
- **[CKO]**: Chapter 27, especially Section 27.3
- RC servo tutorial from Sparkfun
- `AD.h` from the `C:/ECE118/include` directory
- `LED.h` from the `C:/ECE118/include` directory
- `RC_Servo.h` from the `C:/ECE118/include` directory
- Appendix A on LDO regulators from Lab 2

Prelab

Turn in:

- a schematic showing how you will connect the potentiometer while ensuring the Uno32 input always stays between 0V and 3.3V
- a block diagram of the servo setup, as described in Part 0

Bumps and Road Hazards

Bumps and Road Hazards: The servo has three wires: power, ground, and control. One should connect to ground, one to a maximum of 5V, and one carries the command signal. Do *not* connect the servo to a higher voltage.[a]

The Uno32 pins can be damaged by voltages above 3.3V. Verify that your potentiometer divider and anything else you connect to the Uno32 never exceeds that limit, even under likely user error.

Look at the servo control output from the Uno32 on the o'scope *before* connecting the servo. You should see a square pulse with a high time no greater than about 2ms. If you do not see that, do not connect the servo.

The Uno32 stack and the servo must share a common ground back to the power supply.

[a] This needs to be a regulated 5V, either directly from the ATX or through an LDO regulator, connecting more than 5V to an RC servo will destroy it.

Outline

Create a new MPLAB X project for this part and examine the `RC_Servo.h` interface before wiring the final circuit. The servo is a simple electromechanical device containing a motor, gearbox, and internal position feedback. It expects a repeated pulse-width command, with a pulse of about 1.5ms corresponding to the center position.

Read the potentiometer through an analog input, map that reading across the useful servo range, and display a scaled version of the same input on the Uno32 LED bar. The servo should move through a substantial portion of its travel, on the order of about $\pm 40^\circ$ to $\pm 60^\circ$ from center. Do *not* change the `MINPULSE` or `MAXPULSE` values in the provided header.

After basic control is working, explore the servo behavior. What is the minimum change in pulse width that causes visible motion? What angular range does the servo achieve? What is its maximum angular speed, and what is the slowest smooth motion you can command? You may find that direct software-generated test commands are better for these experiments than using the potentiometer.

Lab Report: Include annotated scope traces of the servo control output, relevant code snippets, your block diagram, and a short write-up of what you observed about the servo's range, resolution, and speed.

Part 2 – Unidirectional Drive of a DC Motor

Overview

In this part you will drive a small DC motor in one direction using the DS3658 high-current peripheral driver. You will use a software PWM module to control motor speed based on the potentiometer input and again mirror the scaled input on the LED bar.

Reference Material

- ECE118 Uno32 I/O board documentation
- ECE118 DS3658 documentation
- **[CKO]**: Chapter 8
- `AD.h` from the `C:/ECE118/include` directory
- `LED.h` from the `C:/ECE118/include` directory
- `PWM.h` from the `C:/ECE118/include` directory
- Appendix A on LDO regulators from Lab 2

Prelab

Turn in:

- a schematic for the potentiometer input that keeps the Uno32 within its 0–3.3V input range
- a block diagram of the DS3658 and DC motor setup
- the initialization code you expect to need for the software modules in this part

Bumps and Road Hazards



Bumps and Road Hazards: You will need to make a dedicated power connector for the DS3658. Color-code the wires, use heat shrink on both ends, and triple-check the polarity. The motor's high side should be sourced directly from the Uno32 power-distribution board.

The Uno32 pins can be damaged by voltages above 3.3V. Check the potentiometer wiring before connecting it to the board.

Look at the PWM output from the Uno32 on the o'scope *before* connecting the DS3658. You should see a PWM waveform whose duty cycle changes with the potentiometer. If not, stop and debug first.

Before driving the motor, make sure the DS3658 board is configured in clamped mode. If it is not, inductive kickback can permanently damage the board.

The Uno32 stack and the DS3658 must share a common ground back to the power supply.

Outline

Use the potentiometer as the speed command input and generate a PWM signal from the Uno32 to control the DS3658. The driver can sink current but does not source it, so the motor must be connected with the DS3658 on the low side and the high side of the motor connected directly to the supply.

Scale the potentiometer reading so that 0V corresponds to no LEDs and 0% duty cycle, while 3.3V corresponds to the full LED bar and 100% duty cycle. Use a clean connector strategy between the Uno32 and the DS3658 rather than a fragile nest of jumper wires.

Once the system is working, run the motor at approximately 20% and 80% duty cycle and capture scope traces of the motor voltage. Identify the drive interval, inductive kickback region, clamping behavior, and any other interesting features. It is often helpful to capture the PWM control waveform at the same time for comparison.

Lab Report: Include annotated scope traces of the PWM output and the motor-voltage waveforms at roughly 20% and 80% duty cycle, relevant code snippets, your block diagram, and a short explanation of what you observed.

Part 3 – Snubbing the Inductive Kickback

Overview

In this part you will continue using the DC motor and DS3658, but now the focus shifts to the inductive kickback produced by the motor winding. You will compare different diode and Zener-diode snubbing arrangements and examine how those choices affect the motor waveform, decay time, and usable PWM operating range.

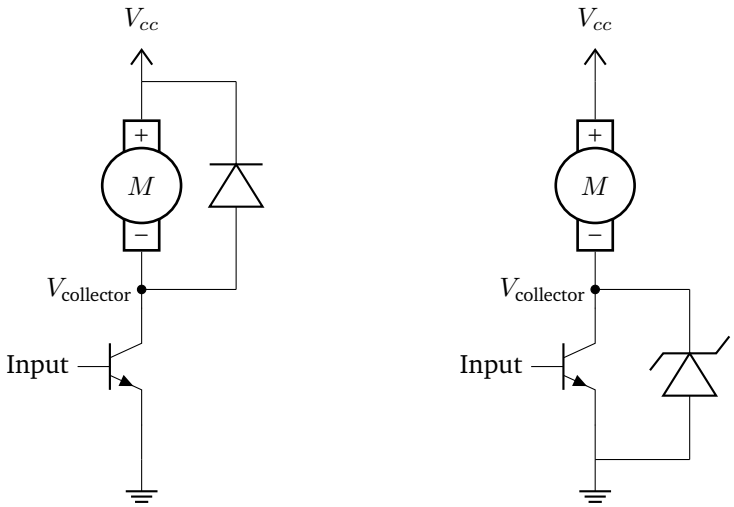


Figure 2: Snubbing topologies for low-side drive using diodes and zeners

Reference Material

- ECE118 Uno32 I/O board documentation
- ECE118 DS3658 documentation
- **[CKO]**: Chapters 22, 23, 24, 26, and 27
- `AD.h` from the `C:/ECE118/include` directory
- `LED.h` from the `C:/ECE118/include` directory
- `PWM.h` from the `C:/ECE118/include` directory

Prelab

Turn in a block diagram of the setup you will use, with everything labeled clearly. Also identify which of the snubbing schematics discussed in the lab corresponds to operating the DS3658 in clamped mode.

Bumps and Road Hazards

Bumps and Road Hazards: All of the warnings from Part 2 still apply here.

Because you will be changing snubbing components, it is especially important to make *all* circuit changes with power off.

The Zener diode is not a normal diode. Make sure you can distinguish the Zener from the standard diode and place each one in the correct circuit.

To measure the waveform across the motor, do *not* connect the oscilloscope ground clip to the collector node of the DS3658. That can short through the scope ground. Instead, measure relative to ground and reconstruct the motor waveform from that measurement.

Outline

Keep the hardware and software arrangement from Part 2, but now compare multiple snubbing configurations (see Fig. 2). You will explore how different diode placements alter the kickback peak and the current-decay behavior when the drive transistor turns off.

Study the motor waveform for each snubbing arrangement at an intermediate duty cycle, a very low duty cycle, and a very high duty cycle. For each case, identify and quantify the kickback peak and the decay time. Compare the results across the different circuits and explain why the behavior changes.

You should also investigate the operating limits of the PWM drive for each snubbing arrangement. What frequency and duty-cycle ranges remain useful? Under what conditions is motor speed approximately linear with duty cycle?

Finally, add load torque to the motor by lightly gripping the shaft and then increasing the load. Observe how the waveform changes and think about why that happens.

Lab Report: Include well-labeled waveform captures for the different snubbing arrangements, identify the kickback peaks and decay times, and explain what you learned about diode placement, PWM limits, and motor loading.

Part 4 – Bidirectional Control of a DC Motor

Overview

In the previous two parts you could only spin the motor in one direction. In this part you will control a DC motor in both directions using an H-bridge. Speed will still come from the potentiometer, but direction will now be determined by a switch.

Reference Material

- ECE118 Uno32 I/O board documentation
- ECE118 L298 dual H-bridge documentation
- **[CKO]**: Chapters 22, 23, 24, 26, and 27
- `AD.h` from the `C:/ECE118/include` directory
- `LED.h` from the `C:/ECE118/include` directory
- `PWM.h` from the `C:/ECE118/include` directory
- Appendix A on hooking up a switch

Prelab

Turn in a block diagram of the setup you will use, with everything labeled clearly, including the initialization code you expect to need.

Bumps and Road Hazards



Bumps and Road Hazards: You will need a proper power connector for the L298 module. Color-code the wires, insulate the joints, and triple-check the polarity.

The Uno32 pins can be damaged by voltages above 3.3V. Check all input wiring before connection.

Look at the Uno32 outputs on the o'scope *before* connecting it to the L298. You should see the expected PWM and direction behavior.

Only change wiring with the power off.

The Uno32 stack and the L298 must share a common ground back to the power supply. Power to the L298 should **always** run through the Power Distribution Board (lowest level of the stack).

Outline

Create a new MPLAB X project that controls the motor bidirectionally in response to the switch and potentiometer. The L298 module requires an enable signal, one or more direction inputs, and an appropriate motor supply.

Your implementation should satisfy the following requirements:

1. 0V at the potentiometer corresponds to 0% PWM duty cycle.
2. 3.3V at the potentiometer corresponds to 100% PWM duty cycle.
3. The switch determines motor direction.
4. The LEDs indicate both speed and direction.

For the LED display, solve the mapping cleanly. If you find yourself writing a very large `switch` statement or a long chain of repetitive `if` statements, step back and look for a better bit-manipulation approach.

Once the basic system works, determine the useful frequency limits of operation and compare them with the earlier DC motor parts. If the limits differ, explain why. Repeat some tests under added shaft load and note what changes.



Lab Report: Include a picture of the setup, relevant code snippets, your block diagram, and a detailed explanation of how you handled speed control, direction control, LED indication, and any operating limits you discovered.

Part 5 – Control of a Stepper Motor

Overview

In this part you will write software to control a stepper motor directly through an H-bridge. You will experiment with full-step, wave, and half-step drive patterns and determine how fast the motor can be started and run without losing steps.

Reference Material

- ECE118 Uno32 I/O board documentation
- ECE118 L298 dual H-bridge documentation
- **[CKO]**: Chapter 26
- `Stepper.h` from the `C:/ECE118/include` directory

Prelab

Turn in:

- a block diagram of the complete setup, including the initialization code you expect to need
- state-machine diagrams for half-step, full-step, and wave-drive sequencing

Bumps and Road Hazards

Bumps and Road Hazards: You will need a power connector for the L298. Color-code the wires, insulate the joints, and triple-check the polarity.

The stepper motor has four wires corresponding to the two coil ends. Use a multimeter or the datasheet to determine the coil pairings before applying power.

The Uno32 pins can be damaged by voltages above 3.3V.

Look at the Uno32 outputs on the o'scope *before* connecting the L298. If the sequence is not correct on the scope, do not connect the driver.

Only change wiring with power off, and keep a common ground between the Uno32 and the L298.

Outline

A stepper motor advances by a fixed angle when its coils are energized in the proper sequence. In this part you will directly command those coil states through the L298 rather than relying on a dedicated stepping board.

Begin with the provided `Stepper.c/.h` module in full-step mode and make sure you understand how the `#define` options select the drive method. Then extend your experiments to wave drive and half-step drive. Only one drive-mode `#define` should be active at a time.

For each drive mode, determine:

- the maximum step rate from which the motor can start reliably
- the maximum running step rate that does not lose steps
- whether those rates change when you add load torque by hand

One simple way to detect lost steps is to mark the shaft, rotate a full turn in one direction and then back again, and see whether the marker returns to its original orientation.

If you want an extra challenge, see whether you can intentionally operate in the pull-in region without losing steps. Read the stepper-motor discussion in CKO first if that term is unfamiliar.



Lab Report: Include the key code snippets, the state diagrams for the different drive methods, and a short discussion of the step-rate limits you found and how they changed with drive mode and loading.

Part 6 – Stepper Motor Using a Dedicated Driver Board

Overview

In this final part you will drive a stepper motor using the DRV8811 (or A4988) dedicated driver board rather than sequencing the H-bridge outputs yourself. This lets you compare direct coil control against a purpose-built stepper driver and examine how current-limit and microstepping settings affect behavior.

Reference Material

- ECE118 Uno32 I/O board documentation
- ECE118 DRV8811 stepper board documentation
- **[CKO]**: Chapter 26
- `Stepper.h` from the lab/course directory

Prelab

Turn in a block diagram of the setup you will use, with everything labeled clearly, including the initialization code you expect to need.

Bumps and Road Hazards

Bumps and Road Hazards: You will need a proper power connector for the DRV8811. Color-code the wires, insulate the joints, and triple-check the polarity.

The stepper still has four coil leads, so identify the coil pairings before connecting the motor.

The Uno32 pins can be damaged by voltages above 3.3V.

Look at the Uno32 outputs on the o'scope *before* connecting the DRV8811. You should see the expected stepping behavior.

Only change wiring with the power off, and maintain a common ground between the Uno32 and the DRV8811.

Outline

Using the `Stepper.c/.h` module, configure the software for the dedicated driver mode and connect the motor to the DRV8811. In this configuration the driver board handles the coil sequencing for you, so your software only needs to provide step, direction, and enable control.

Determine the maximum start rate and the maximum sustainable running rate without losing steps. As before, test whether load torque changes those limits. Then compare those results against the direct-drive approach from Part 5.

Also explore the DRV8811 configuration options. The jumpers on the board may change current limits or step modes, and those settings can substantially alter the feel and limits of the motor.

Lab Report: Include the key code snippets and a short discussion of what you discovered about step-rate limits, loading effects, and the practical differences between direct H-bridge drive and a dedicated stepper-driver board.

Checkoff and Time Tracking and Final Lab Report

Checkoffs will be tracked electronically. One of the tutors or TAs will ask you to demonstrate your work, possibly answer questions about your implementation, and explain what you observed. The checkoff will be recorded when you demonstrate the required functionality. You should check off each part before moving on to the next one, but if you cannot find a tutor or TA immediately, there is no penalty for working out of order as long as you can still return to a working earlier section.

Keep a rough time log for each part, including work done outside the lab and time spent physically in the lab. The point is not bureaucratic precision; it is to help you and us understand where the time really goes in a motor-control lab with substantial hardware bring-up.

Lab Report: Include a table of your time logs and any observations about what worked well and what did not.

Time Spent out of Lab	Time Spent in Lab	Lab Part – Description
		Part 0 – Preparation for Lab
		Part 1 – Driving an RC Servo
		Part 2 – Unidirectional Drive of a DC Motor
		Part 3 – Snubbing the Inductive Kickback
		Part 4 – Bidirectional Control of a DC Motor
		Part 5 – Control of a Stepper Motor
		Part 6 – Stepper Motor Using a Dedicated Driver Board

Also include any overall comments about the lab, what you learned, what you found difficult, and what you would improve.

Self- and Partner Reflection

After you have completed the lab, give some thought to how you worked as a team, and how you feel that you supported your partner. Fill out the Self- and Partner Reflection form: `https://forms.gle/BdVY1wKwdzzek8Mz8` to rate yourself and your partner in terms of how well you each contributed to the lab.*

Appendix A – Hooking Up a Switch

Switches appear throughout mechatronic systems, and the Uno32 needs a clean logic-level signal when a switch changes state. For a single switch, we will focus on either an SPST switch or an SPDT switch (see Fig. 3).



Figure 3: SPST and SPDT switches

*This will not be used for grading, but is very helpful for the teaching staff to figure out who needs more help.

An SPST switch is a two-terminal device that is open when released and electrically continuous when closed. An SPDT switch has three terminals: common, normally open, and normally connected. Throwing the switch changes which of the latter two is connected to common.

In both cases, the goal is to produce a signal at the Uno32 pin that transitions cleanly between 0V and 3.3V. A standard way to do this is to connect the switch common to ground, route the switched node to the input pin, and use a pull-up resistor to 3.3V. An indicator LED and resistor may also be added if you want visible feedback, but they are optional (see Fig. 4).

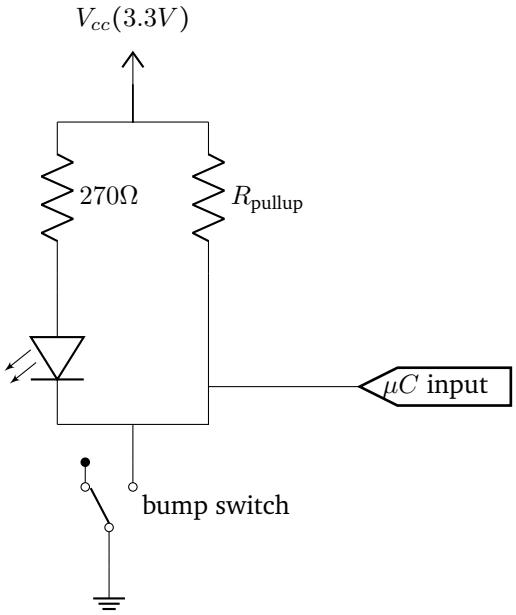


Figure 4: SPDT switch hooked to Roach Bump Sensor

When the switch is open, little or no current flows and the pin reads a logic high near 3.3V. When the switch closes, the node is pulled to ground and the pin reads a logic low near 0V. This is an “active-low” switch input and is extremely common in embedded hardware.